

KELAS INDUSTRI	MODUL PEMBELAJARAN KELAS INDUSTRI	Pertemuan 02
-----------------------	--	-------------------------

Satuan Pendidikan	:	Sekolah Menengah Kejuruan (SMK)
Kompetensi Keahlian	:	Rekayasa Perangkat Lunak (RPL / PPLG)
Kelas / Semester	:	XI atau XII / Semester Ganjil
Pertemuan Ke	:	02 (Dua) dari Seri Next.js Dasar sampai Mahir
Alokasi Waktu	:	4 JP (4 × 45 Menit = 180 Menit)
Topik Pertemuan	:	Lanjutan Tugas 1: Styling Tailwind CSS, Komponen Reusable, Interaktivitas, dan Dynamic Routing
Pola Pembelajaran	:	100% Praktik Kejuruan Project-Based Learning Teori 35% : Praktik Terbimbing 65%

I. CAPAIAN PEMBELAJARAN (LEARNING OUTCOMES)

1. Siswa mampu merombak dan mempercantik antarmuka project Website Profil Siswa dari Tugas 1 menggunakan utility-first class Tailwind CSS.
2. Siswa mampu menerapkan tata letak Flexbox, Grid, dan prinsip Mobile-First Responsive Design pada halaman web profil siswa.
3. Siswa mampu merefaktor kode tampilan monolitik menjadi komponen terpisah yang reusable dengan tipe data props berbasis TypeScript interface.
4. Siswa memahami arsitektur rendering Next.js App Router: perbedaan, batasan, serta alur kerja Server Component dan Client Component.
5. Siswa mampu menambahkan fitur interaktif pada project Tugas 1 menggunakan direktif 'use client', hook useState, dan penanganan event onClick.
6. Siswa mampu mengimplementasikan Dynamic Routing menggunakan folder segment [id] untuk menampilkan rincian proyek secara fleksibel.
7. Siswa mampu menangani parameter rute asinkron (params) dan parameter query pencarian (searchParams) standar Next.js 15+.
8. Siswa mampu memisahkan lapisan data lokal (data/proyek.ts) dari antarmuka visual dan menghubungkannya ke halaman katalog serta detail proyek.
9. Siswa mampu menangani kondisi data tidak ditemukan dengan fungsi notFound() dan merancang halaman kustom app/not-found.tsx.

II. KONTEKS INDUSTRI & RELEVANSI KARIR

Di dunia industri software development, seorang developer jarang membangun project dari nol setiap hari. Mayoritas pekerjaan nyata di industri adalah melanjutkan, mengembangkan, dan mempercantik kode yang sudah ada (existing codebase). Melanjutkan project 'Profil Siswa' dari Tugas 1 melatih siswa membiasakan diri dengan alur kerja profesional: melakukan refactoring kode menjadi komponen reusable, menata styling responsif standar industri dengan Tailwind CSS, memisahkan Server Component demi efisiensi loading, serta mengalirkan data dinamis secara aman. Keterampilan ini merupakan standar minimum rekrutmen Junior Front-End Developer di industri teknologi saat ini.

Platform & Referensi Belajar Standar Industri

Platform	URL / Tautan	Deskripsi & Kegunaan
Dokumentasi Resmi Next.js - Dynamic Routes	https://nextjs.org/docs/app/building-your-application/routing/dynamic-routes	Rujukan standar pengelolaan segment rute dinamis [id] dan penanganan asynchronous params di Next.js 15+
Dokumentasi Resmi Next.js - Server & Client Components	https://nextjs.org/docs/app/building-your-application/rendering/server-components	Panduan resmi batas pemisahan Server vs Client Components dan optimasi bundle size browser
Dokumentasi Resmi Tailwind CSS	https://tailwindcss.com/docs	Dokumentasi lengkap utility class untuk Spacing, Typography, Flexbox, Grid, dan Breakpoint responsif
React Official Docs - State & Event Handling	https://react.dev/learn/state-a-components-memory	Konsep fundamental pengelolaan memori interaktif antarmuka menggunakan hook useState

III. KEBUTUHAN PERANGKAT & TOOLS

No.	Kebutuhan	Detail	Keterangan
1	Project Profil Siswa Tugas 1	Folder project Next.js 'profil-siswa' dari Pertemuan 01 yang sudah memiliki struktur app/, page.tsx, dan layout.tsx	Wajib - fondasi kelanjutan
2	Node.js v18.18+ / v20+	Runtime JavaScript yang telah digunakan di Pertemuan 01 untuk menjalankan development server Next.js	Wajib - runtime sistem
3	Tailwind CSS	Sudah terpasang secara otomatis di project Tugas 1 (konfigurasi Tailwind bawaan create-next-app)	Sudah tersedia dari Tugas 1

No.	Kebutuhan	Detail	Keterangan
4	Visual Studio Code	Editor kode dengan ekstensi Tailwind CSS IntelliSense dan TypeScript ESLint aktif	Direkomendasikan
5	Browser Modern	Google Chrome / Firefox / Edge dengan Developer Tools (F12) untuk pengujian mode responsif smartphone	Wajib - pengujian multi-device

IV. PROJECT PERTEMUAN INI

Nama Project	Pengembangan Lanjutan Website Profil Siswa & Portfolio (Lanjutan Tugas 1)
Teknologi	Next.js 15 + TypeScript + Tailwind CSS (App Router, Server & Client Components)
Tingkat Kesulitan	Menengah Kejuruan (Fase F SMK RPL / PPLG)
Output Akhir	Website Profil Siswa & Portfolio terintegrasi dengan tata letak responsif mobile-first, memiliki navigasi Navbar & Footer global, interaktivitas client state, katalog proyek dengan filter kategori berbasis URL, dynamic routing /proyek/[id], dan penanganan error 404 yang aman.
Roadmap Lanjutan	Project ini akan dihubungkan dengan database cloud Supabase (PostgreSQL) untuk fitur penyimpanan data dinamis dan form kontak pada Pertemuan 03.

Melanjutkan dan menyempurnakan project Website Profil Siswa yang telah dibuat pada Pertemuan 01. Website yang sebelumnya masih berupa teks mentah polos kini ditransformasikan menjadi aplikasi web portofolio profesional: antarmuka di-styling rapi dengan Tailwind CSS responsif, kode dipecah menjadi komponen reusable, ditambahkan interaktivitas tombol apresiasi dan menu mobile, serta diintegrasikan dengan Dynamic Routing [id] dan data lokal proyek yang terhubung ke halaman detail dan 404 kustom.

V. ALUR PEMBELAJARAN 2 JP (90 MENIT)

No.	Durasi	Mode / Format	Aktivitas Pembelajaran
1	15 Menit	Instruktur - Telaah Kode & Arah Pembelajaran	Segmen 1: Pembukaan, Evaluasi Tugas 1 & Peta Pengembangan (15 Menit) - Instruktur membuka kelas dan mengulas hasil project Website Profil Siswa dari Tugas 1. - Tunjukkan masalah nyata: tampilan masih polos seperti dokumen teks biasa dan belum responsif di smartphone. - Jelaskan roadmap hari ini: 'Hari ini kita tidak membuat project baru dari nol! Kita melanjutkan project Tugas 1 kemarin, mempercantik dengan Tailwind CSS, memecahnya menjadi komponen, menambah klik interaktif, dan membuat halaman detail proyek dinamis.' - Tegaskan bahwa Tailwind CSS sudah terpasang sejak Tugas 1 sehingga tidak perlu diinstal ulang.
2	35 Menit	Instruktur - Penjelasan Konsep, Analogi & Live Demo	Segmen 2: Teori 1-4: Styling Tailwind, Komponen, dan Server vs Client (35 Menit) - Jelaskan konsep Utility Class Tailwind: spacing, typography, colors, flexbox, grid, dan prinsip Mobile-First. - Analogi LEGO untuk Component Architecture: mengapa kode dipecah ke folder components/ dan cara membuat props ber-interface TypeScript. - Analogi Dapur vs Meja Restoran untuk Server Component (0 KB JS) vs Client Component ('use client'). - Demokan cara kerja useState untuk tombol hitungan interaktif dan event onClick.

No.	Durasi	Mode / Format	Aktivitas Pembelajaran
3	35 Menit	Instruktur - Alur Data & Demo Rute Asinkron	Segmen 3: Teori 5-9: Dynamic Routing, Params, Data Lokal & 404 (35 Menit) - Jelaskan masalah routing statis jika memiliki puluhan proyek portofolio. - Papar visual konsep Dynamic Segment [id] dan penanganan async params standar Next.js 15+. - Jelaskan struktur data lokal data/proyek.ts dan cara komponen membaca data tersebut. - Bandingkan [id] (menentukan item proyek) dengan searchParams (filter kategori seperti ? category=web). - Jelaskan peran fungsi notFound() dan pembuatan custom error 404 di app/not-found.tsx.
4	80 Menit	Siswa Coding Mandiri + Asistensi Instruktur	Segmen 4: Praktik Terbimbing - Refactoring & Integrasi Project Tugas 1 (80 Menit) - Siswa membuka project profil-siswa dari Tugas 1 di VS Code dan menjalankan npm run dev. - Step 1 & 2: Merombak app/layout.tsx dan app/page.tsx dengan styling Tailwind CSS responsif. - Step 3: Membuat folder components/ dan membangun Navbar, Footer, Badge, serta CardProyek. - Step 4: Menambahkan komponen interaktif CounterApresiasi.tsx dan MobileMenu.tsx ber-directive 'use client'. - Step 5 & 6: Menyiapkan data/proyek.ts dan membuat halaman katalog app/proyek/page.tsx dengan filter searchParams. - Step 7 & 8: Membuat rute detail dinamis app/proyek/[id]/page.tsx dan app/not-found.tsx. - Instruktur berkeliling memeriksa jalannya kode dan membantu mengatasi error sintaks.

No.	Durasi	Mode / Format	Aktivitas Pembelajaran
5	15 Menit	Uji Pengujian & Evaluasi Instruktur	Segmen 5: Uji Responsivitas Chrome DevTools & Penutup (15 Menit) - Siswa menguji website di Chrome DevTools (Ctrl + Shift + M) pada lebar 375px (smartphone) dan desktop. - Uji simulasi: klik filter kategori, buka rute detail proyek, dan coba ketik ID 999 untuk menguji halaman 404. - Pemberian arahan pengerjaan Tugas Mandiri untuk diselesaikan di rumah. - Preview Pertemuan 03: 'Minggu depan kita akan menghubungkan portofolio ini dengan database cloud Supabase!'

VI. MATERI TEORI & KONSEP KUNCI

1. Styling & Responsive Design dengan Tailwind CSS (Mobile-First)

Pengantar: Pada Tugas 1, kita telah memiliki website Profil Siswa dengan 5 halaman, namun tampilannya masih sangat polos. Tailwind CSS sudah terpasang secara bawaan sejak pembuatan project di Modul 1, sehingga kita tidak perlu menginstal apapun lagi.

Materi & Penjelasan: Tailwind CSS menggunakan konsep Utility-First. Kita merangkai tampilan langsung di atribut `className` menggunakan class-class fungsional kecil:

- Spacing: `p-4` (padding 16px), `px-4` (padding horizontal), `py-2` (padding vertikal), `m-4` (margin), `gap-4` (jarak antar elemen).
- Tipografi: `text-sm` (14px), `text-base` (16px), `text-xl` (20px), `text-3xl` (30px), `font-bold` (700), `text-slate-800`.
- Tata Letak: Flexbox (`flex`, `items-center`, `justify-between`) untuk 1 dimensi, dan CSS Grid (`grid`, `grid-cols-1`, `md:grid-cols-2`) untuk 2 dimensi.
- Prinsip Mobile-First: Seluruh class tanpa awalan berlaku untuk layar ponsel pintar. Tambahkan awalan breakpoint untuk layar yang lebih lebar: `sm:` (`>=640px`), `md:` (`>=768px`), `lg:` (`>=1024px`).

Contoh Kode Sederhana:

Perhatikan penulisan kartu profil responsif berikut:

Breakpoint Tailwind	Lebar Layar Minimal	Perilaku Layout pada Project Profil
Default (HP)	< 640px	Kartu dan navigasi tersusun vertikal (1 kolom) agar pas dengan layar smartphone.
sm: (Tablet Kecil)	≥ 640px	Ruang padding membesar dan teks mulai berukuran sedang.
md: (Tablet / iPad)	≥ 768px	Grid bertransformasi menjadi 2 kolom berdampingan dan menu desktop mulai aktif.
lg: (Laptop)	≥ 1024px	Kontainer maksimal 5xl berada di tengah layar dengan margin otomatis (mx-auto).

Struktur kartu profil adaptif berbasis Tailwind CSS Utility Classes

```
// Contoh tata letak kartu responsif: 1 kolom di HP, 2 kolom di tablet/laptop
<div className="grid grid-cols-1 md:grid-cols-2 gap-6 p-6 bg-white rounded-xl shadow-md">
  <div>
    <h2 className="text-2xl font-bold text-slate-900">Ahmad Siswa RPL</h2>
    <p className="text-slate-600 text-sm mt-2 leading-relaxed">
      Fokus pada rekayasa antarmuka web modern menggunakan Next.js App Router.
    </p>
  </div>
  <div className="flex items-center justify-center bg-slate-50 rounded-lg p-4">
    <span className="text-xs font-semibold text-blue-600">Portofolio Siswa RPL</span>
  </div>
</div>
```

2. Arsitektur Komponen & Props dengan TypeScript Interface

Pengantar: Di Tugas 1, kita menuliskan seluruh isi halaman secara menumpuk di file page.tsx. Cara ini membuat kode sulit dirawat dan tidak reusable.

Materi & Penjelasan: Analogi balok LEGO sangat tepat menggambarkan komponen. Sebuah website tersusun dari balok-balok kecil seperti Navbar, Footer, Badge, dan Kartu Proyek. Jika kita butuh 10 kartu proyek, kita tidak menyalin tag HTML 10 kali, melainkan membuat satu komponen kartu dan memanggilnya 10 kali.

- Props (Properties): Cara mengirimkan data dari komponen induk ke komponen anak.

- TypeScript Interface: Kontrak tertulis untuk memastikan data props yang dikirim memiliki tipe yang benar.

Bedah Kode: Perhatikan pembuatan komponen Badge kategori dengan kontrak tipe data TypeScript:

Komponen reusable Badge ber-props TypeScript untuk menandai kategori proyek

```
// components/Badge.tsx
interface BadgeProps {
  label: string;
}

export default function Badge({ label }: BadgeProps) {
  const isWeb = label.toLowerCase() === "web";
  const colorStyle = isWeb
    ? "bg-blue-100 text-blue-700 border-blue-200"
    : "bg-emerald-100 text-emerald-700 border-emerald-200";

  return (
    <span className={`inline-block px-2.5 py-0.5 text-xs font-semibold rounded-full border ${colorStyle}`}>
      {label}
    </span>
  );
}
```

3. Server Component vs Client Component (Batas Rendering)

Pengantar: Mengapa Next.js App Router memisahkan komponen menjadi Server dan Client?

Materi & Penjelasan:

- Server Component (Default): Seluruh file di folder app/ diproses di server Node.js. Hasil yang dikirim ke browser pengguna adalah 100% HTML murni (0 KB JavaScript). Ini membuat website memuat dengan kecepatan maksimal dan sangat aman untuk data rahasia.
- Client Component: Komponen yang membutuhkan interaksi pengguna langsung (klik mouse, ketikan keyboard) atau memori browser (state). Dideklarasikan dengan menuliskan 'use client' di baris paling pertama file.
- Analogi Restoran: Server Component adalah Dapur Restoran (koki mengolah bahan masakan tanpa terlihat tamu), sedangkan Client Component adalah Meja Tamu (pelanggan berinteraksi menuang saus atau menambah garam secara langsung).

Aturan Industri: Biarkan halaman utama dan layout tetap berupa Server Component. Jadikan Client Component hanya bagian-bagian kecil yang memang butuh tombol klik.

Kebutuhan Fungsional	Server Component	Client Component ('use client')
Menampilkan teks profil & data	Sangat Direkomendasikan (Cepat)	Bisa, namun boros bundle JS
Mengirim file JS ke browser	0 KB (No Byte)	Mengirimkan bundle JS hidrasi
Menangani klik tombol (onClick)	Tidak Bisa Digunakan	Wajib Digunakan
Menggunakan hook useState	Tidak Bisa Digunakan	Wajib Digunakan
Keamanan data & kredensial	Sangat Aman (di server)	Rentan Bocor ke browser klien

4. Interaktivitas Antarmuka: useState dan Event Handling

Pengantar: Halaman web profil siswa pada Tugas 1 bersifat pasif. Bagaimana jika pengunjung ingin memberikan nilai apresiasi terhadap karya kita?

Materi & Penjelasan: Variabel JavaScript biasa (seperti `let nilai = 0`) tidak bisa digunakan karena React tidak mendeteksi perubahannya untuk merender ulang layar. Untuk data interaktif yang dapat berubah, kita menggunakan hook `useState` dari React.

- Sintaks: `const [apresiasi, setApresiasi] = useState(0);`
- Bedah: `apresiasi` adalah penyimpanan nilai saat ini, `setApresiasi` adalah fungsi resmi pengubah nilai, dan `0` adalah nilai awal.
- Event Handling: Di React ditulis camelCase (`onClick`). Kita memberikan fungsi panah untuk mengeksekusi penambahan nilai.

Komponen interaktif Client Component dengan pengelolaan state useState

```
// components/CounterApresiasi.tsx
"use client";

import { useState } from "react";

export default function CounterApresiasi() {
  const [skor, setSkor] = useState(0);

  return (
    <div className="inline-flex items-center gap-3 bg-white px-4 py-2 rounded-xl border border-slate-200 shadow-sm">
      <span className="text-xs font-semibold text-slate-600">Apresiasi:</span>
      <span className="font-mono font-bold text-blue-600 min-w-[1.5rem] text-
```

```

center">{skor}</span>
    <button
      onClick={() => setSkor(skor + 1)}
      className="px-3 py-1 bg-blue-600 hover:bg-blue-700 text-white font-bold text-xs
rounded-lg transition"
    >
      + Beri Apresiasi
    </button>
  </div>
);
}

```

5. Dynamic Routing Menggunakan Segment Folder [id]

Pengantar: Pada Tugas 1, kita membuat halaman /proyek secara statis. Jika siswa kelak memiliki 100 karya proyek, apakah harus membuat 100 folder manual? Tentu tidak efisien.

Materi & Penjelasan: Di Next.js App Router, tanda kurung siku [id] pada nama folder menciptakan rute dinamis (wildcard segment). Satu file cetakan app/proyek/[id]/page.tsx dapat menangani URL /proyek/1, /proyek/2, hingga /proyek/100 secara otomatis.

Alur Kerja Penangkapan Parameter:

1. Pengguna mengetik URL /proyek/1 di browser.
2. Next.js mengenali folder segment dinamis [id].
3. Next.js mengemas nilai 1 ke dalam objek parameter (params).
4. File page.tsx membaca parameter ID tersebut dan mencari data proyek yang sesuai.

6. Standar Penanganan Parameter Asinkron (params & searchParams)

Pengantar: Standar terbaru Next.js 15+ mengubah cara membaca parameter rute menjadi asinkron.

Materi & Penjelasan:

- **params (Rute Dinamis):** Menentukan entitas data spesifik, contohnya /proyek/1. Parameter ini bertipe Promise sehingga wajib diambil menggunakan kata kunci async dan await params.
- **searchParams (Query URL):** Memberikan kriteria filter atau pencarian pada halaman yang sama, contohnya /proyek?category=web. Menggunakan searchParams memungkinkan filter dibagikan sebagai link URL yang tetap aktif saat dibuka oleh orang lain.

Pembacaan parameter asinkron params pada Server Component Next.js 15+

```

// Contoh penanganan parameter asinkron standar Next.js 15+
interface HalamanDetailProps {
  params: Promise<{ id: string }>;
}

export default async function DetailProyek({ params }: HalamanDetailProps) {

```

```
// Wajib menggunakan await sebelum mengakses properti id pada Next.js 15+
const { id } = await params;
return <h1 className="text-xl font-bold">Menampilkan Proyek ID: #{id}</h1>;
}
```

7. Pengelolaan Data Lokal Terstruktur (data/proyek.ts)

Pengantar: Sebelum menghubungkan database Supabase pada Pertemuan 03, kita memisahkan data dari tampilan menggunakan file TypeScript lokal.

Materi & Penjelasan: Data diletakkan di dalam folder data/proyek.ts. Komponen tampilan akan mengimpor data ini dan menampilkannya menggunakan perulangan .map(). Jika suatu saat ada data proyek baru yang ingin ditambahkan, kita cukup memperbarui array data tersebut tanpa menyentuh kode tampilan HTML.

Koleksi data lokal TypeScript terstruktur untuk portofolio siswa

```
// data/proyek.ts
export interface ProyekItem {
  id: string;
  judul: string;
  kategori: "Web" | "Mobile" | "IoT";
  deskripsiSingkat: string;
  deskripsiLengkap: string;
  teknologi: string[];
}

export const daftarProyek: ProyekItem[] = [
  {
    id: "1",
    judul: "Sistem Presensi Siswa Barcode",
    kategori: "Web",
    deskripsiSingkat: "Aplikasi absensi harian siswa berbasis scan barcode dan geolokasi.",
    deskripsiLengkap: "Aplikasi ini dirancang untuk mengatasi antrean absensi pagi di sekolah...",
    teknologi: ["Next.js", "TypeScript", "Tailwind CSS"]
  },
  {
    id: "2",
    judul: "E-Kantin Sekolah Digital",
    kategori: "Mobile",
    deskripsiSingkat: "Pemesanan menu kantin sehat bebas antrean berbasis saldo digital.",
    deskripsiLengkap: "Siswa dapat memesan makanan dari ruang kelas saat jam istirahat...",
    teknologi: ["React Native", "Tailwind CSS", "Node.js"]
  }
]
```

```
}  
];
```

8. Penanganan Kondisi Error 404 dengan notFound()

Pengantar: Apa yang terjadi jika pengunjung iseng mengetik URL `/proyek/999` padahal data proyek nomor 999 tidak pernah ada?

Materi & Penjelasan: Tanpa penanganan error, website akan menampilkan halaman kosong putih atau crash error runtime. Next.js menyediakan fungsi resmi `notFound()` dari modul `next/navigation`. Saat fungsi ini dipanggil, Next.js secara otomatis mengalihkan tampilan ke file kustom `app/not-found.tsx` yang menyajikan pesan informatif dan tombol kembali ke katalog.

Tampilan kustom error 404 ramah pengguna pada Next.js App Router

```
// app/not-found.tsx  
import Link from "next/link";  
  
export default function NotFound() {  
  return (  
    <main className="min-h-[60vh] flex items-center justify-center text-center px-4">  
      <div className="max-w-md space-y-4">  
        <span className="text-6xl font-extrabold text-blue-600 font-mono">404</span>  
        <h1 className="text-2xl font-bold text-slate-900">Karya Proyek Tidak  
Ditemukan</h1>  
        <p className="text-slate-600 text-sm">Data proyek dengan ID tersebut tidak  
terdaftar di portofolio.</p>  
        <Link href="/proyek" className="inline-block px-5 py-2.5 bg-blue-600 text-white  
text-sm font-semibold rounded-lg hover:bg-blue-700 transition">  
          Kembali ke Daftar Proyek  
        </Link>  
      </div>  
    </main>  
  );  
}
```

9. Integrasi Menyeluruh ke Project Profil Siswa Tugas 1

Pengantar: Seluruh konsep di atas bukan materi yang terpisah-pisah, melainkan satu alur utuh yang langsung disuntikkan ke dalam project Profil Siswa dari Tugas 1.

Arsitektur Integrasi:

1. Root Layout (`app/layout.tsx`): Memasang Navbar dan Footer global serta font Google Inter.
2. Beranda (`app/page.tsx`): Di-styling dengan hero section Tailwind dan preview keahlian.
3. Halaman Proyek (`app/proyek/page.tsx`): Menampilkan kartu proyek dari `data/proyek.ts` dengan filter query URL.

4. Halaman Detail (app/proyek/[id]/page.tsx): Membaca ID dinamis dan menampilkan deskripsi mendalam.
5. Halaman Tentang (app/tentang/page.tsx): Ditambahkan komponen interaktif counter apresiasi.
6. Halaman 404 (app/not-found.tsx): Menjaga keamanan rute dari ID yang tidak valid.

VII. PANDUAN PRAKTIK STEP-BY-STEP (HANDS-ON LAB)

Step 1 - Membuka Project Tugas 1 & Verifikasi Tailwind CSS

Buka terminal VS Code, arahkan ke folder project profil-siswa yang telah dibuat pada Tugas 1, lalu jalankan development server. Pastikan server aktif tanpa error:

```
# Masuk ke direktori project Tugas 1
cd profil-siswa

# Jalankan development server
npm run dev

# Buka browser di http://localhost:3000
# Tailwind CSS sudah aktif secara bawaan sejak pembuatan project Tugas 1
```

Step 2 - Merombak app/layout.tsx Menjadi Layout Global Responsif

Buka file app/layout.tsx dari Tugas 1. Ganti isinya agar memiliki Navbar terstruktur dengan Link dan Footer terpadu:

```
// app/layout.tsx
import type { Metadata } from 'next';
import Link from 'next/link';
import './globals.css';

export const metadata: Metadata = {
  title: 'Website Profil Siswa & Portfolio - Next.js',
  description: 'Portofolio karya teknologi siswa SMK RPL',
};

export default function RootLayout({
  children,
}): {
  children: React.ReactNode;
} {
  return (
    <html lang="id">
      <body className="min-h-screen flex flex-col bg-slate-50 text-slate-900 font-sans">
        {/* Navigasi Utama Global */}
        <header className="sticky top-0 z-50 bg-white/90 backdrop-blur-md border-b
```

```

border-slate-200">
    <div className="max-w-5xl mx-auto px-4 h-16 flex items-center justify-
between">
        <Link href="/" className="font-extrabold text-lg text-slate-900 tracking-
tight">
            Profil<span className="text-blue-600">Siswa</span>
        </Link>
        <nav className="flex items-center gap-6 text-sm font-medium text-
slate-600">
            <Link href="/" className="hover:text-blue-600 transition">Beranda</Link>
            <Link href="/tentang" className="hover:text-blue-600
transition">Tentang</Link>
            <Link href="/keahlian" className="hover:text-blue-600
transition">Keahlian</Link>
            <Link href="/proyek" className="hover:text-blue-600 transition">Proyek</
Link>
            <Link href="/kontak" className="hover:text-blue-600 transition">Kontak</
Link>
        </nav>
    </div>
</header>

{/* Konten Halaman Dinamis */}
<div className="flex-1 max-w-5xl mx-auto px-4 py-8 w-full">
    {children}
</div>

{/* Footer Global */}
<footer className="border-t border-slate-200 bg-white py-6 text-center text-xs
text-slate-500">
    <p>Project Pembelajaran Next.js App Router – SMK RPL / PPLG</p>
</footer>
</body>
</html>
);
}

```

Step 3 - Mempercantik Beranda (app/page.tsx) dengan Hero Section Responsif

Buka file app/page.tsx dan ubah menjadi beranda modern dengan tata letak Mobile-First:

```

// app/page.tsx
import Link from 'next/link';

export default function Beranda() {
    return (

```

```

<main className="py-8 md:py-16">
  <div className="space-y-6 max-w-2xl">
    <span className="inline-block px-3 py-1 bg-blue-100 text-blue-700 text-xs font-
semibold rounded-full">
      Siswa SMK Rekayasa Perangkat Lunak
    </span>
    <h1 className="text-3xl sm:text-4xl lg:text-5xl font-extrabold text-slate-900
tracking-tight leading-tight">
      Halo, Selamat Datang di <span className="text-blue-600">Profil &
Portofolio</span> Saya
    </h1>
    <p className="text-slate-600 text-base sm:text-lg leading-relaxed">
      Project ini merupakan kelanjutan dari Tugas 1 yang telah dikembangkan dengan
Tailwind CSS,
      arsitektur komponen modular, interaktivitas state, dan dynamic routing
Next.js App Router.
    </p>
    <div className="flex flex-wrap gap-4 pt-2">
      <Link
        href="/proyek"
        className="px-6 py-3 bg-blue-600 hover:bg-blue-700 text-white text-sm font-
semibold rounded-lg shadow-sm transition"
      >
        Lihat Koleksi Proyek &rarr;
      </Link>
      <Link
        href="/tentang"
        className="px-6 py-3 bg-white hover:bg-slate-100 text-slate-700 border
border-slate-300 text-sm font-semibold rounded-lg transition"
      >
        Tentang Saya
      </Link>
    </div>
  </div>
</main>
);
}

```

Step 4 - Membuat Folder components/ dan Komponen CardProyek.tsx Reusable

Buat folder baru bernama components di root project. Buat file Badge.tsx dan CardProyek.tsx dengan props TypeScript:

```

// components/CardProyek.tsx
import Link from 'next/link';
import Badge from './Badge';

```

```

export interface CardProyekProps {
  id: string;
  judul: string;
  kategori: string;
  deskripsiSingkat: string;
  teknologi: string[];
}

export default function CardProyek({
  id,
  judul,
  kategori,
  deskripsiSingkat,
  teknologi,
}: CardProyekProps) {
  return (
    <div className="bg-white rounded-xl border border-slate-200 p-6 flex flex-col justify-between shadow-sm hover:shadow-md hover:border-blue-300 transition">
      <div className="space-y-3">
        <div className="flex items-center justify-between">
          <Badge label={kategori} />
          <span className="text-xs font-mono text-slate-400">#{id}</span>
        </div>
        <h3 className="text-lg font-bold text-slate-900">{judul}</h3>
        <p className="text-sm text-slate-600 line-clamp-2 leading-relaxed">{deskripsiSingkat}</p>
      </div>
      <div className="mt-6 pt-4 border-t border-slate-100 flex items-center justify-between">
        <div className="flex flex-wrap gap-1">
          {teknologi.slice(0, 2).map((t) => (
            <span key={t} className="text-[11px] bg-slate-100 text-slate-600 px-2 py-0.5 rounded">{t}</span>
          ))}
        </div>
        <Link href={` /proyek/${id}`} className="text-xs font-semibold text-blue-600 hover:underline">
          Detail &rarr;
        </Link>
      </div>
    </div>
  );
}

```


Step 5 - Menyiapkan Data Lokal Proyek (data/proyek.ts)

Buat folder data di root project, lalu buat file proyek.ts berisi 4 data proyek representatif jurusan:

```
// data/proyek.ts
export interface ProyekItem {
  id: string;
  judul: string;
  kategori: 'Web' | 'Mobile' | 'IoT';
  deskripsiSingkat: string;
  deskripsiLengkap: string;
  teknologi: string[];
}

export const daftarProyek: ProyekItem[] = [
  {
    id: '1',
    judul: 'Sistem Informasi Presensi Barcode',
    kategori: 'Web',
    deskripsiSingkat: 'Pencatatan absensi siswa harian berbasis scan barcode kartu pelajar dan validasi GPS.',
    deskripsiLengkap: 'Aplikasi ini dibangun untuk mengatasi antrean presensi gerbang sekolah pada jam masuk pagi. Wali kelas dapat memantau kehadiran siswa secara real-time dari ruang guru.',
    teknologi: ['Next.js', 'TypeScript', 'Tailwind CSS']
  },
  {
    id: '2',
    judul: 'Aplikasi Pemesanan E-Kantin Mobile',
    kategori: 'Mobile',
    deskripsiSingkat: 'Pemesanan menu kantin sekolah sehat bebas antrean dengan pembayaran digital.',
    deskripsiLengkap: 'Siswa dapat memesan menu sarapan dan makan siang sebelum jam istirahat berbunyi, sehingga mengurangi kerumunan antrean di stan kantin sekolah.',
    teknologi: ['React Native', 'Tailwind CSS', 'Node.js']
  },
  {
    id: '3',
    judul: 'Monitoring Smart GreenHouse IoT',
    kategori: 'IoT',
    deskripsiSingkat: 'Pemantauan kelembaban tanah dan penyiram otomatis kebun hidroponik sekolah.',
    deskripsiLengkap: 'Menggunakan mikrokontroler ESP32 dan protokol MQTT untuk mengontrol pompa nutrisi tanaman sayur hidroponik secara otomatis saat tanah mengering.',
    teknologi: ['Next.js', 'MQTT', 'Arduino C++']
  },
]
```

```

{
  id: '4',
  judul: 'Portal Tracer Study & Alumni SMK',
  kategori: 'Web',
  deskripsiSingkat: 'Platform pelacakan keterserapan kerja lulusan SMK di dunia usaha dan industri.',
  deskripsiLengkap: 'Memudahkan alumni mendata status pekerjaan, wirausaha, atau studi lanjut (BMW) dengan visualisasi statistik grafik yang informatif.',
  teknologi: ['Next.js', 'Tailwind CSS', 'TypeScript']
}
];

```

Step 6 - Memperbarui Halaman Proyek (app/proyek/page.tsx) dengan Filter searchParams

Buka file app/proyek/page.tsx dari Tugas 1, ganti kodenya agar menyajikan katalog kartu dan filter query kategori:

```

// app/proyek/page.tsx
import CardProyek from '@components/CardProyek';
import { daftarProyek } from '@data/proyek';
import Link from 'next/link';

interface ProyekPageProps {
  searchParams: Promise<{ category?: string }>;
}

export default async function ProyekPage({ searchParams }: ProyekPageProps) {
  const { category } = await searchParams;
  const filtered = category
    ? daftarProyek.filter((p) => p.kategori.toLowerCase() === category.toLowerCase())
    : daftarProyek;

  const categories = ['Semua', 'Web', 'Mobile', 'IoT'];

  return (
    <main className="py-6 space-y-6">
      <div>
        <h1 className="text-3xl font-extrabold text-slate-900">Daftar Karya & Proyek Siswa</h1>
        <p className="text-slate-600 text-sm mt-1">Pilih kategori di bawah untuk menyaring portofolio karya kejuruan:</p>
      </div>

      {/* Filter Kategori berbasis query URL */}
      <div className="flex flex-wrap gap-2">
        {categories.map((cat) => {

```

```

        const isAll = cat === 'Semua';
        const href = isAll ? '/proyek' : `/proyek?category=${cat.toLowerCase()}`;
        const isActive = isAll ? !category : category?.toLowerCase() ===
cat.toLowerCase();
        return (
            <Link
                key={cat}
                href={href}
                className={`px-4 py-1.5 rounded-lg text-xs font-semibold transition ${
                    isActive ? 'bg-blue-600 text-white shadow-sm' : 'bg-white text-
slate-600 border border-slate-200 hover:bg-slate-100'
                }`}
            >
                {cat}
            </Link>
        );
    }}}
</div>

{/* Grid Kartu */}
<div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6 pt-2">
    {filtered.map((item) => (
        <CardProyek key={item.id} {...item} />
    ))}
</div>
</main>
);
}

```

Step 7 - Membuat Dynamic Route Detail Proyek (app/proyek/[id]/page.tsx)

Buat folder [id] di dalam folder app/proyek/, kemudian buat file page.tsx. Komponen ini membaca params dan memanggil notFound() jika ID salah:

```

// app/proyek/[id]/page.tsx
import { notFound } from 'next/navigation';
import Link from 'next/link';
import Badge from '@components/Badge';
import { daftarProyek } from '@data/proyek';

interface DetailProps {
    params: Promise<{ id: string }>;
}

export default async function DetailProyekPage({ params }: DetailProps) {
    const { id } = await params;

```

```

const proyek = daftarProyek.find((p) => p.id === id);

if (!proyek) {
  notFound();
}

return (
  <main className="py-6 space-y-6">
    <Link href="/proyek" className="text-xs font-semibold text-blue-600
hover:underline inline-block">
      &larr; Kembali ke Daftar Proyek
    </Link>

    <article className="bg-white rounded-2xl border border-slate-200 p-6 md:p-8
shadow-sm space-y-6">
      <div className="space-y-2 border-b border-slate-100 pb-4">
        <div className="flex items-center gap-3">
          <Badge label={proyek.kategori} />
          <span className="text-xs font-mono text-slate-400">Route ID: #{id}</span>
        </div>
        <h1 className="text-2xl sm:text-3xl font-extrabold text-
slate-900">{proyek.judul}</h1>
      </div>

      <div className="space-y-2">
        <h2 className="text-sm font-bold text-slate-900 uppercase tracking-
wider">Latar Belakang & Solusi:</h2>
        <p className="text-slate-600 text-sm sm:text-base leading-
relaxed">{proyek.deskripsiLengkap}</p>
      </div>

      <div className="pt-4 border-t border-slate-100 space-y-2">
        <h3 className="text-xs font-bold text-slate-700">Teknologi yang Digunakan:</
h3>
        <div className="flex flex-wrap gap-2">
          {proyek.teknologi.map((t) => (
            <span key={t} className="px-3 py-1 bg-slate-100 text-slate-700 text-xs
rounded-md font-medium">
              {t}
            </span>
          ))}
        </div>
      </div>
    </article>
  </main>

```

```
);  
}
```

Step 8 - Membuat Halaman 404 Kustom (app/not-found.tsx)

Buat file not-found.tsx di dalam folder app/. Halaman ini akan otomatis muncul jika pengguna mengetikkan rute atau ID yang tidak ada:

```
// app/not-found.tsx  
import Link from 'next/link';  
  
export default function NotFound() {  
  return (  
    <main className="min-h-[50vh] flex items-center justify-center text-center px-4 py-12">  
      <div className="space-y-4 max-w-md">  
        <span className="text-6xl font-extrabold text-blue-600 font-mono">404</span>  
        <h1 className="text-2xl font-bold text-slate-900">Halaman / Proyek Tidak Ditemukan</h1>  
        <p className="text-slate-600 text-sm leading-relaxed">  
          Maaf, data karya proyek dengan ID yang kamu masukkan tidak tersedia di sistem portofolio.  
        </p>  
        <Link  
          href="/proyek"  
          className="inline-block px-5 py-2.5 bg-blue-600 text-white rounded-lg text-xs font-semibold hover:bg-blue-700 transition"  
        >  
          Kembali ke Daftar Proyek  
        </Link>  
      </div>  
    </main>  
  );  
}
```

Step 9 - Memperbarui Halaman Tentang Saya dengan Interaktivitas Apresiasi

Buat komponen components/CounterApresiasi.tsx (Client Component), lalu panggil di dalam file app/tentang/page.tsx dari Tugas 1:

```
// app/tentang/page.tsx  
import CounterApresiasi from '@components/CounterApresiasi';  
  
export default function TentangSaya() {  
  return (  
    <main className="py-6 space-y-6 max-w-3xl">  
      <h1 className="text-3xl font-extrabold text-slate-900">Tentang Saya</h1>  
    </main>  
  );  
}
```

```

    <p className="text-slate-600 leading-relaxed">
      Saya adalah siswa SMK jurusan Rekayasa Perangkat Lunak (RPL). Halaman ini
      merupakan pengembangan lanjutan
      dari Tugas 1 yang kini telah dilengkapi interaktivitas antarmuka dan penataan
      komponen yang rapi.
    </p>
    <div className="bg-blue-50 border border-blue-200 rounded-xl p-6 flex flex-col
sm:flex-row items-center justify-between gap-4">
      <div>
        <h2 className="font-bold text-slate-900 text-sm">Beri Dukungan untuk Karya
        Siswa</h2>
        <p className="text-xs text-slate-600 mt-1">Klik tombol apresiasi di samping
        untuk mencoba fitur interaktivitas Client Component:</p>
      </div>
      <CounterApresiasi />
    </div>
  </main>
);
}

```

VIII. TUGAS MANDIRI (DILANJUTKAN DI RUMAH)

Pengembangan Mandiri Portfolio Siswa (Lanjutan Tugas 1)

Lanjutkan pengembangan website Profil Siswa dari Tugas 1 di rumah. Tambahkan minimal 2 proyek baru buatanmu sendiri ke dalam data/proyek.ts, rapikan halaman Keahlian dan Kontak dengan Tailwind CSS responsif, serta pastikan seluruh link rute dinamis dan penanganan error 404 berjalan dengan sempurna.

Deliverables Wajib Dikumpulkan:

- Repository GitHub yang berisi update lanjutan project profil-siswa dari Tugas 1 dengan riwayat commit rapi.
- Dokumentasi README.md yang memuat penjelasan penambahan fitur: Tailwind styling, daftar komponen baru di folder components/, dan rute dinamis /proyek/[id].
- Tangkapan layar (screenshot) pengujian responsif pada layar smartphone (375px) menggunakan Chrome DevTools.
- Tangkapan layar pengujian rute 404 saat mengakses URL proyek yang tidak ada (contoh: /proyek/999).

Tantangan Opsional (untuk yang lebih cepat):

- Tambahkan kotak pencarian judul proyek (search input) berbasis Client Component pada halaman katalog proyek.
- Tambahkan efek animasi transisi hover halus pada kartu proyek (hover:-translate-y-1 hover:shadow-lg).

- Terapkan tombol filter kategori tambahan di halaman Beranda untuk menampilkan proyek unggulan (featured projects).

IX. RUBRIK ASESMEN & EVALUASI

No.	Komponen Penilaian	Indikator Ketercapaian	Bobot	Skor Maks
1	Styling Tailwind CSS & Mobile-First	Menerapkan class utilitas Tailwind pada layout Tugas 1 dengan rapi, proporsional, dan bebas overflow horizontal pada layar smartphone (375px).	25%	100
2	Arsitektur Komponen & TypeScript Interface	Memisahkan kode ke folder components/ (Navbar, Badge, CardProyek) dengan validasi props TypeScript interface tanpa tipe any.	20%	100
3	Pemisahan Server vs Client Component	Menggunakan 'use client' dan useState secara tepat hanya pada komponen interaktif (CounterApresiasi), layout utama tetap Server Component.	20%	100
4	Dynamic Routing & Integrasi Data Lokal	Rute dinamis /proyek/[id] membaca async params dengan benar, data lokal data/proyek.ts terhubung, dan query searchParams berfungsi.	25%	100
5	Penanganan Error 404 & Kerapian Kode	Fungsi notFound() aktif saat ID salah menuju app/not-found.tsx kustom, kode bersih, dan lolos uji kompilasi npm run dev tanpa error.	10%	100

X. ROADMAP KELAS INDUSTRI - PERTEMUAN SELANJUTNYA

Setiap project yang dibuat di pertemuan ini adalah fondasi dari project minggu berikutnya. Berikut gambaran perjalanan belajar yang akan ditempuh:

Minggu	Tema / Project	Output	Status
1	Pengenalan & Dasar-Dasar Next.js - App Router, Routing, Navigasi Link	Website multi-halaman 'Profil Siswa' dengan navigasi Link	Selesai
2	Styling Tailwind, Komponen Reusable, Interaktivitas & Dynamic Routing	Website Profil & Portofolio Siswa terintegrasi dengan dynamic route [id] dan filter	Pertemuan ini
3	Integrasi Cloud Database Supabase (PostgreSQL) - CRUD Data Portofolio	Koleksi proyek tersimpan di cloud database dan dapat ditambah secara online	Berikutnya
4	Server Actions, Form Mutasi, & Proteksi Halaman Admin	Halaman dashboard admin terproteksi untuk mengelola proyek portofolio	Akan datang
5	Deployment Produksi ke Vercel & Optimasi SEO Metadata	Website live di internet dengan domain custom dan performa skor tinggi	Akan datang

XI. DAFTAR REFERENSI

Seluruh materi dan panduan praktik dalam modul ini disusun berdasarkan dokumentasi resmi dan standar industri teknologi terkini (Next.js 15 App Router, React 19, dan Tailwind CSS):

1. Next.js 15 Official Documentation - Dynamic Routes: <https://nextjs.org/docs/app/building-your-application/routing/dynamic-routes>
2. Next.js 15 Official Documentation - Server Components: <https://nextjs.org/docs/app/building-your-application/rendering/server-components>
3. Next.js 15 Official Documentation - Client Components: <https://nextjs.org/docs/app/building-your-application/rendering/client-components>
4. Next.js 15 Official Documentation - Functions notFound: <https://nextjs.org/docs/app/api-reference/functions/not-found>
5. Tailwind CSS Documentation - Responsive Design: <https://tailwindcss.com/docs/responsive-design>
6. React Documentation - useState Hook: <https://react.dev/reference/react/useState>

**Mengetahui,
Guru Instruktur Sekolah**

**Disetujui Oleh,
Mentor Praktisi Industri**

(_____)
NIP / ID:

(_____)
NIP / ID: